SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I JEROLD SAMUEL J (192571052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
JEROLD SAMUEL J

Engineer to Excel

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and

Assessment tool - 3

Jerold Samuel.J
192571052

Scenario Based Task

1) Parsing Table conflict in LL(1)

In an LL(1) parser the parsing table is constructed using the first & follow sets.

* First gives the terminals that can appear at the beggining of the string derived from a production

* Follow gives the terminals that can appear immediately after a non-terminal

These sets are used to decide which production should be placed in each parsing table cell.

The common reason for this conflicts are:

1) Left recursion
2) common prefixes
3) Ambigous grammar
4) overlapping First / Follow sets.

Since an LL(1) parser must make a decision using only one input symbol, multiple production in one cell make the parser non-deterministic

Decision:

The grammar should be modified by:

* Eliminating left recursion.
* Applying left factoring
* Removing ambiguity
* Ensuring First & Follow sets do not create conflicts.

This produces a conflicts-free LL(1) grammar

2) Incorrect parse Tree generation.

For the expression:

id + id - id

The syntax analyzer cretes a parse tree according to the grammar rule. The structure of this tree determines the order in which the operators are evaluated

If the grammar does not speciby operator associativity, the expression may be interpreted incorrectly as; id + (id-id) instead of, (id + id) - id.

Addition & subtraction should normally be evaluated from left to right, so they require left associativity.

Decision:

The grammar should enforce left associativity

$E \rightarrow E+T \mid E-T \mid T$

$T \rightarrow id$

This ensures that the expression is grouped from left to right and produces the correct evaluation.

3) Invalid Expression handling:

The given expression is:

a + * b.

The lexical analyzer can identify

a → identifier

+ → operator

* → operator

b → identifier.

Although every token is individually valid, the sequence of is syntactically invalid

After the + operator the grammar expects on operand such as identifier or number. Instead of operator appears.

a + * b

Invalid position.

The syntax analyzer cannot construct a valid parse tree because the tokens sequence violates the grammar.

(i) Predictive parsing.
(ii) Shift-Reduce Parsing.

Decision:

The parser should use error-recovery technique such as

* Panic-mode recovery
* Phrase-level recovery
* Follow-set synchronization.

These techniques allow the parser to report meaningful errors & continue analyzing the remaining input instead of stopping immediately.

4) Ambiguity in conditional statement

The grammar is.

$S \rightarrow$ if E then s

$S \rightarrow$ if E then S else s

$S \rightarrow a$

This problem is that an else can potentially belong to more than one if.

For example:

if E1 then
    if E2 then
        a
    else
        a

This parser must decide whether the else belong to the inner if or the outer if
This is called dangling else problem.

The two production.

S → if E then S

S → if E then S else S

have a common prefix: if E then S

Therefore The parser may encounter a conflict while constructing the parsing table.

Effect:

The ambiguity can produces.

* parsing-table conflicts.
* Multiple possible parse tree
* Non-deterministic parsing.

Decision:

S → M | U

M → if E then M else M | a

U → if E then S
    | if E then M else U

5) operator precedence conflicts.

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id.$

The input is : id + id * id.

The grammar is ambiguous

two possible interpretations are:

(id + id) * id

(or)

id + (id * id)

The correct interpretations is

id + (id * id)

because multiplication has higher precedence than addition.

Decision.

$E \rightarrow E+T / T$

$T \rightarrow T * F / F$

$F \rightarrow id.$

(i) Here, * has high precedence than +.

(ii) * and + are left associative.

Therefore, id + id * id.

the parse becomes id + (id * id)

This ensures the correct evaluation order.

Alternatively, Parser generator can use precedence & associativity declarasion to resolve such conflicts automatically.